Anlamın Sınırları: Wittgenstein'dan Ontolojiye ve Domain-Driven Design'a

Ömer Faruk Yavuz

October 2026

Özet

Aynı kelimenin farklı topluluklarda farklı nesnelere karşılık gelmesi, geç Wittgenstein'ın anlamın kullanımdan doğduğu tespitinin yazılımdaki görünümüdür. Tek bir kanonik şema bu farkı sessizce siler; Palantir Ontology ve Domain-Driven Design bu soruna verilmiş iki ayrı cevaptır. Ontology farkı ortak bir nesne katmanında eritir ve çeviriyi bir kez, verinin platforma alındığı noktada yapar. Domain-Driven Design ise farkı korur ve çeviriyi her topluluk sınırına, açıkça adlandırılmış bir katman olarak yerleştirir. Taktiksel düzeyde iki yaklaşım büyük ölçüde örtüşür: nesne tipleri Entity'ye, fonksiyonlar Domain Service'e yakındır. Ayrışma, sınır çizen araçlarda yoğunlaşır; Context Map ve Aggregate'in Ontology'de doğrudan karşılığı bulunmaz. Domain-Driven Design'ın taktiksel kararlarının tamamı, Entity davranışından Aggregate sınırına kadar, domain uzmanına söylenen bir cümlenin sınanmasına dayanır; bu, bir kuralın uygulamasının kuraldan değil pratikten çıktığı tespitinin mühendislik karşılığıdır. İki yaklaşım arasındaki asıl fark etkinlikte değil, hangi topluluğun dilinin geçerli olacağı kararının görünür kalıp kalmadığındadır.

Giriş

Sorun ve Amaç

Bu çalışma tek bir soruyu izleyerek felsefeden yazılım mühendisliğine uzanan bir güzergâhı incelemektedir. Soru şudur: *aynı kelime farklı topluluklarda farklı anlamlara geldiğinde, bu farklılık tek bir bilgi sisteminde nasıl temsil edilir?*

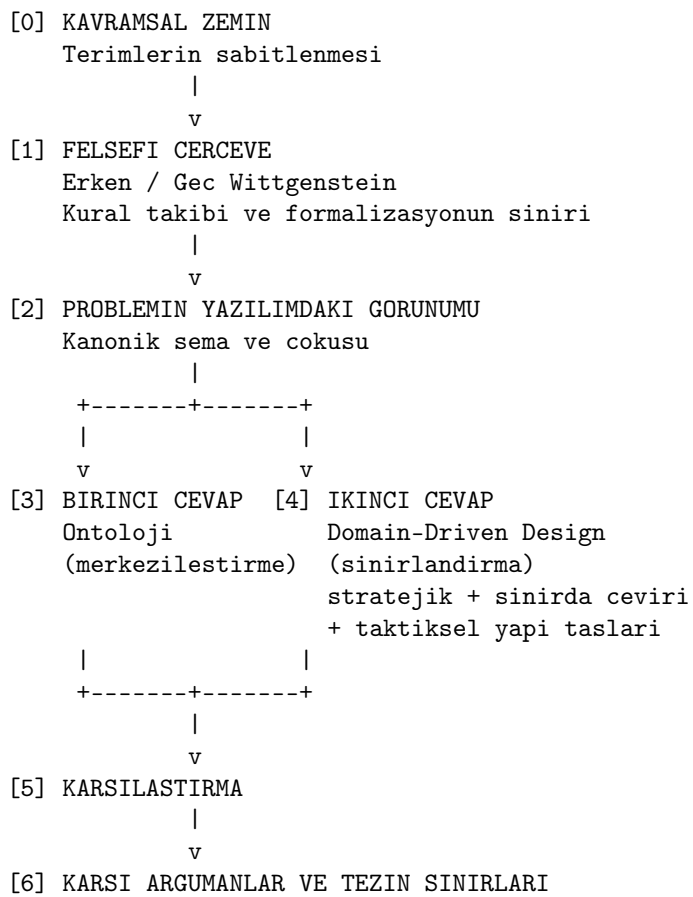
Söz konusu sorun önce felsefi bir tespit olarak ortaya çıkmış, ardından bir mühendislik problemine dönüşmüş ve nihayetinde birbirinden ayrışan iki kurumsal cevap üretmiştir. Çalışmanın amacı bu iki cevabı — Palantir Ontology ile Domain-Driven Design'ı — ortak felsefi zeminleri üzerinden karşılaştırmak ve aralarındaki farkın teknik değil kurumsal niteliğini göstermektir.

Tez

Çalışmanın savunduğu tez üç önermeden oluşmaktadır.

1. Her iki yaklaşım da aynı felsefi tespitten hareket etmektedir: kavramlar topluluğa görelidir ve tek bir kanonik şemaya sıkıştırılamaz.
2. Verdikleri cevaplar zıt yönlüdür. Domain-Driven Design farkı koruyup sınırlar arasına açık bir çeviri katmanı yerleştirir; Ontology farkı ortak bir nesne katmanında eritir.
3. Aradaki en kritik ayrım performans veya ifade gücü değil, *gücün görünürlüğüdür*. Domain-Driven Design hangi topluluğun dilinin geçerli olacağını açık bir tasarım kararına dönüştürür; Ontology aynı kararı ortadan kaldırmaz, yalnızca altyapı konfigürasyonuna gömerek görünmez kılar.

Yol Haritası



Yöntem ve Sınırlılıklar

Çalışmanın kanıt tabanında, açıkça beyan edilmesi gereken bir asimetri bulunmaktadır.

Kaynak asimetrisi. Domain-Driven Design tarafı, öğretisini doğrudan ortaya koyan birincil ve ikincil metinlerden okunmuştur: Evans'ın kurucu kitabı, Vernon'un uygulama kitabı ve Ürkmez'in bu öğretiyi Türkçede sistematik biçimde işleyen on yazılık serisi. Serinin ilk dört yazısı stratejik tasarımı, beşinci yazısı bağlamlar arası entegrasyonu, son beş yazısı ise taktiksel tasarımı ele almaktadır. Ontology tarafı ise büyük ölçüde ürün dokümantasyonundan, şirket beyanlarından ve üçüncü taraf teknik incelemelerden okunmuştur. Birinci grup

nasıl yapılması gerektiğini anlatan normatif metinlerdir; ikinci grup ise *ürünün nasıl sunulduğunu* gösteren metinlerdir.

Bunun sonucu şudur: karşılaştırma, doğası gereği Domain-Driven Design'ı en olgun hâliyle, Ontology'yi ise beyan edilmiş hâliyle ele almaktadır. Bu eşitsizlik ortadan kaldırılamamıştır; altıncı bölümde açıkça bir karşı argüman olarak ele alınmaktadır.

Ampirik dayanağın yokluğu. Çalışma kavramsal bir analizdir. Vaka incelemesi, ölçüm veya saha gözlemi içermemektedir. Dolayısıyla ileri sürülen tez, bir hipotez olarak okunmalı ve ampirik sınamaya açık kabul edilmelidir.

Felsefi okumanın niteliği. Wittgenstein'a yapılan başvuru araçsaldır, metin şerhi niteliğinde değildir. Amaç, mühendislik tartışmasına kavramsal bir çerçeve kazandırmaktır; Wittgenstein yorumculuğuna katkı sunmak değildir. Çalışma boyunca kullanılan Türkçe terimler için Barışcan'ın *Felsefi Soruşturmalar* çevirisi esas alınmıştır.

Kavramsal Zemin

Aşağıdaki liste hızlı başvuru amaçlıdır. Terimler ilgili bölümlerde ayrıntılı olarak ele alınmaktadır.

Anlam ve Formalizasyon

- **Anlam.** Bir işaretin neye karşılık geldiği. Bu çalışmada işaretin kendisinde bulunan bir özellik değil, kullanımdan doğan bir ilişki olarak ele alınır. *Örnek:* "Yeşil ışık" trafikte geçme izni, projede yönetici onayı, ameliyathanede hastanın stabil olduğu bilgisidir.
- **Özcülük.** Her kavramın, tüm örneklerinde ortak bulunan bir çekirdek niteliği olduğu varsayımı. *Örnek:* "Sandalye" tanımı her sıkılaştırmada bir örneği dışarıda bırakır, her gevşetmede bir yabancıyı içeri alır.
- **Dil oyunu.** Kelimenin, içine gömülü olduğu faaliyet ve o faaliyetin kurallarıyla birlikte oluşturduğu bütün. *Örnek:* Şantiyede "tuğla" bir emirdir, malzeme dersinde bir sınıflandırma birimidir, arkeolojik raporda bir tarihlendirme kanıtıdır.
- **Aile benzerliği.** Ortak tek bir nitelik bulunmamasına karşın örtüşen benzerlik zincirleriyle bir kavramın tutarlı işlemesi. *Örnek:* Maraton, satranç, dart ve otomobil yarışının hepsini kapsayan tek bir "spor" niteliği saptanamaz.
- **Formalizasyon.** Bir pratiğin açıkça yazılmış, sonlu ve makine tarafından işlenebilir kurallara dönüştürülmesi. *Örnek:* Bir tarifte malzemeler ve süreler yazılabilir; "hamur kıvamını alınca" ifadesi yazılamaz.
- **Kural takibi problemi.** Bir kuralın nasıl uygulanacağının kuralın kendisinden türetilemeyeceği tespiti. *Örnek:* "Kütüphanede sessiz olunuz" kuralı, öksürüğün ve sandalye gıcırtısının kapsam içinde olup olmadığını söylemez.
- **Örtük bilgi.** Uygulanan ancak sorulduğunda formüle edilemeyen bilgi. *Örnek:* Deneyimli kaynakçının dikişin tuttuğunu sesinden anlaması.

Modelleme ve Temsil

- **Model.** Bir alanın, belirli bir amaç doğrultusunda seçilmiş yönlerini temsil eden yapı; tanımı gereği eksiktir. *Örnek:* Metro haritası coğrafi olarak yanlıştır ve işlevini tam da bu yanlışlık sayesinde görür.
- **Şema.** Verinin hangi alanlardan oluştuğunu belirleyen biçimsel tanım. *Örnek:* Formda "ikinci uyruk" kutusu yoksa o bilgi sisteme hiç girmez.
- **Kanonik.** Birden fazla aday arasından birinin resmî ve bağlayıcı sayılması. *Örnek:* Uyuşmazlıkta insan kaynaklarının yayımladığı tatil listesinin geçerli sayılması.
- **Ontoloji.** Felsefede varlığın temel kategorilerini inceleyen metafizik dalı; veri alanında bir alandaki nesne tiplerini, özelliklerini ve ilişkilerini tanımlayan yapı.
- **Ham veri.** Yorum katmanı bulunmayan kayıtlar. *Örnek:* Market fişindeki barkod ve tutar, ne alındığını söylemez.
- **Anlam katmanı.** Ham verinin gerçek dünyadaki karşılığını tanımlayan, veriyi değiştirmeyip okunuşunu belirleyen yapı.
- **Uygulamalı soyutlama.** Kâğıt üzerinde kalmayıp çalışan bir sisteme dönüşen ve bu nedenle sınanabilen soyutlama. *Örnek:* Yanlış konumlandırılmış bir kapı, planda görünmez; inşaatta ilk gün kendini gösterir.
- **Dijital ikiz.** Bir kurumun yazılımda birebir karşılığının kurulduğu iddiası. Harita ölçekli sadeleştirmedir ve sadeleştirdiğini beyan eder; ikiz eksiksizlik iddiası taşır.

Topluluk, Sınır ve Güç

- **Topluluk.** Ortak bir işi ortak kurallarla yürüten ve bu nedenle ortak bir kelime dağarcığı geliştiren grup. *Örnek:* Boş bir yatak hemşire için kapasite, muhasebe için gelir kaybıdır.
- **Bağlam.** Bir kavramın belirli bir anlama sabitlendiği alan.
- **Sınır.** İki bağlamın ayrıldığı ve kavramların anlam değiştirdiği hat. Teknik değil dilseldir. *Örnek:* Pasaport kontrol hattının asıl işlevi konum değil, kural değişimidir.
- **Çeviri ve çeviri kaybı.** Bir bağlamın kavramlarının başka bir bağlama dönüştürülmesi ve bu dönüşümde bazı ayırt edici unsurların düşmesi. *Örnek:* "Acil" kaydın sırasıyla "öncelikli" ve "bugün içinde" hâline gelmesi.

- **Asimetri.** Etkinin tek yönlü akması. Değer sıralaması değil, bağımlılık yönüdür. *Örnek:* Ana yüklenicinin kararı taşeronun planını yeniden kurar; tersi olmaz.
- **Müzakere.** Hangi bağlamın kavramının geçerli olacağının açıkça tartışılıp karara bağlanması. *Örnek:* Ortak alan temizliğinin toplantıda karara bağlanması ile bir dairenin sessizce üstlenmesi aynı sonucu üretir; ikincisi tartışılamaz, çünkü hiç kurulmamış gibidir.
- **Sessiz hata.** Sistemin çökmediği, uyarı üretmediği, ancak yanlış çalıştığı durum. *Örnek:* Sabit biçimde şaşan bir terazi, her tartımda tutarlı ve yanlış sonuç verir.

Domain-Driven Design Terimleri

Türkçe karşılıkları yerleşmediği için özgün biçimleriyle bırakılmıştır.

Stratejik terimler

- **Domain.** Organizasyonun varlık nedeni ve bilgisinin bütünü. Banka için bankacılık, havayolu için havacılık.
- **Problem space / Solution space.** Hangi problemin önemli olduğu sorusunun alanı ile bunun yazılımda nasıl karşılanacağı sorusunun alanı.
- **Subdomain.** Domain'in mantıksal parçası; Core, Supporting veya Generic olarak sınıflandırılır. Problem space'e aittir.
- **Bounded Context.** Bir domain modelinin yaşadığı ve kavramların tek anlama sabitlendiği sınır. Solution space'e aittir.
- **Ubiquitous Language.** Bir bağlam içinde domain uzmanları ile geliştiricilerin birlikte kurduğu ve toplantıda, dokümantasyonda ve kodda aynı biçimde kullanılan dil.
- **Domain expert.** İşi herkesten iyi bilen kişi; unvan değil konum.
- **Context Map.** Bağlamların ve aralarındaki ilişkilerin haritası; hedeflenen mimariyi değil mevcut durumu çizer.
- **Anemic Domain Model.** Veri taşıyıp davranış taşımayan modellerin oluşturduğu anti-pattern.
- **Big Ball of Mud.** Sınırların tutarsızlaştığı ve kavramsal düzenin kaybedildiği sistem bölgesi.

Sınır ve entegrasyon terimleri

- **Open Host Service / Published Language.** Üst taraftaki bağlamın tüm tüketicilere sunduğu tek açık protokol ve bu protokolde kullanılan, belgelenmiş ve kararlı ortak dil.
- **Anticorruption Layer (ACL).** Alt taraftaki bağlamın, üst tarafın modelini kendi diline çevirdiği koruma katmanı.
- **Translation Map.** Dış bir temsilin alanlarının yerel modelin kavramlarına eşlenme tablosu; neyin alındığını ve neyin dışarıda bırakıldığını gösterir.
- **Eventual consistency.** Sınırlar arası tutarlılığın aynı anda değil, belirli bir süre içinde sağlanması.

Taktiksel terimler

- **Entity.** Kimliği olan ve zaman içinde değişebilen nesne. Öznitelikleri değil, kimliği onu tanımlar.
- **Value Object.** Kimliği olmayan, yalnızca taşıdığı değerlerle tanımlanan ve oluşturulduktan sonra değişmeyen nesne.
- **Primitive obsession.** Alan kavramlarının anlam taşıyan tipler yerine `string`, `int`, `decimal` gibi çıplak dil tipleriyle temsil edilmesi.
- **Domain Service.** Tek bir Entity ya da Value Object'e ait olmayan, birden fazla nesnenin bilgisini gerektiren, adı domain dilinden gelen ve durum tutmayan işlem.
- **Invariant.** Her an doğru kalması gereken iş kuralı. *Örnek:* Bir siparişin ödenecek tutarı ile kalemlerinin toplamı arasındaki eşitlik.
- **Aggregate.** Bir tutarlılık sınırı: içindeki kuralların her işlemden sonra, tek bir transaction içinde birlikte doğru kalması gereken nesne grubu. Dışarıyla yalnızca kök nesnesi (Aggregate Root) üzerinden iletişim kurar.
- **Repository.** Aggregate'leri saklayıp geri getiren ve dışarıya bir koleksiyon gibi görünen sözleşme.
- **Domain Event.** Bir Aggregate'te gerçekleşmiş, domain uzmanının önemsediği ve geriye dönük değiştirilemeyen olayın kaydı. Geçmiş zamanla adlandırılır.

Terminoloji Notu

Küçük harfle "ontoloji" genel kavramı, "Ontology" Palantir ürününü belirtmektedir. Benzer biçimde "bağlam" genel anlamda, "Bounded Context" Domain-Driven Design'ın teknik terimi olarak kullanılmaktadır. Bu ayrım çalışmanın konusunun doğrudan bir gereğidir: aynı kelimenin iki dil oyununda iki farklı nesneye karşılık gelmesi, incelenen olgunun kendisidir.

Felsefi Çerçeve

Ontoloji Teriminin İki Yaşamı

Ontoloji terimi iki ayrı alanda, iki farklı anlamda kullanılmaktadır ve aralarındaki bağ türeyimseldir.

Felsefi kullanım

Felsefede ontoloji, "ne vardır?" sorusunu ele alan metafizik dalıdır. Varlığın en temel kategorilerini inceler: dünya nihai olarak nelerden oluşmaktadır, hangi şeyler gerçekten vardır ve hangileri türevdir? Nesneler mi vardır yoksa süreçler mi? Bir orman var mıdır, yoksa yalnızca ağaçlar mı vardır ve orman bizim koyduğumuz bir etiket midir?

Felsefi ontoloji, kısaca, gerçekliğin envanterini ve o envanterin kategorilere ayrılma biçimini sorgular.

Teknik kullanım

Veri alanındaki anlam daha somuttur: bir ontoloji, belirli bir alandaki nesne tiplerini, bunların özelliklerini ve aralarındaki ilişkileri açıkça tanımlayan yapıdır.

Ham veri, tablo düzeyinde birbirinden kopuk hücrelerden oluşur. Ontoloji ise bu hücrelerin üzerine şu iddiayı yerleştirir: burada gerçekte bir `Kayıt` nesnesi, bir `Kullanıcı` nesnesi ve bir `İndirme` olayı bulunmaktadır; ve bunlar arasında tanımlı ilişkiler vardır.

Bu haliyle ontoloji, "satır ve sütun" temsilinden "nesne ve ilişki" temsiline geçiştir: verinin, makinenin depolama biçimiyle değil gerçek dünyadaki karşılığıyla modellenmesi girişimidir.

Erken Wittgenstein: Resim Kuramı

Tractatus Logico-Philosophicus'un konumu şöyle özetlenebilir: dil dünyanın bir resmidir, her anlamlı önerme bir olgu durumuna karşılık gelir ve doğru analiz edildiğinde tek bir mantıksal iskelet ortaya çıkar. Dünya, şeylerin değil olguların toplamıdır.

Bu konum formal, sonlu ve tam bir temsil idealini savunur. Modern veri modellemesinin örtük felsefesi büyük ölçüde budur; bu tespit çalışmanın ilerleyen bölümlerinde belirleyici olacaktır.

Geç Wittgenstein: Dil Oyunları ve Aile Benzerliği

Felsefi Soruşturmalar, bu konumun yazarı tarafından terk edildiği metindir. Merkezî iddia şudur:

> Bir kelimenin anlamı, dildeki kullanımıdır.

Bu iddiaya göre bir kavramın çekirdek tanımı yoktur. Kavramı anlamlı kılan şey, belirli bir topluluğun onu belirli pratikler içinde nasıl kullandığıdır: kimi kapsadığı, kimi dışladığı ve hangi eylemi tetiklediği. Wittgenstein bu bütünlüğe **dil oyunu** adını verir; kelime, kurallarıyla birlikte bir faaliyetin içine gömülüdür. Faaliyet değiştiğinde anlam da değişir, çünkü anlam zaten faaliyetin içinde bulunmaktaydı.

İkinci kavram **aile benzerliğidir**. "Oyun" kelimesinin kapsadığı örneklerin tümünde bulunan tek bir özellik saptanamaz. Satranç, futbol, solitaire ve çocukların yakalamacası arasında kimisi rekabetçidir kimisi değildir; kimisi katı kurallıdır kimisi gevşektir. Ortak bir öz yoktur; örtüşen benzerlik zincirleri vardır. Bu tespit, "yeterince kesin bir tanım yapılırsa kavram yakalanır" varsayımına doğrudan bir itirazdır.

Kural Takibi ve Formalizasyonun Sınırı

Geç Wittgenstein'ın en radikal iddiası çoğunlukla atlanan bir noktadadır. İddia yalnızca "anlam kullanımdadır" değil, "anlam tam olarak formalize edilemez"dir.

Kural takibi argümanının vardığı sonuç şudur: hiçbir kural kendi uygulamasını belirlemez. Bir kuralın nasıl uygulanacağı, kuralın kendisinden türetilemez; her formalizasyon, kendisinin nasıl okunacağını belirleyemeyen bir pratiğe yaslanır.

Bu sonuç, çalışmanın ilerleyen bölümlerinde ele alınacak gerilimin kaynağıdır: incelenen mühendislik cevaplarının her ikisi de, tam olarak anlamı formalize etme girişimidir. Dolayısıyla ikisi de aynı felsefi itirazın hedefindedir ve aralarındaki fark, itirazı ne ölçüde karşıladıklarında değil, itiraza rağmen neyi feda ettiklerinde aranmalıdır.

Problemin Yazılımdaki Görünümü

Aynı Kelime, Farklı Nesne

Bir bankada "müşteri" kelimesinin dört ayrı birimdeki kullanımı incelendiğinde şu tablo ortaya çıkmaktadır:

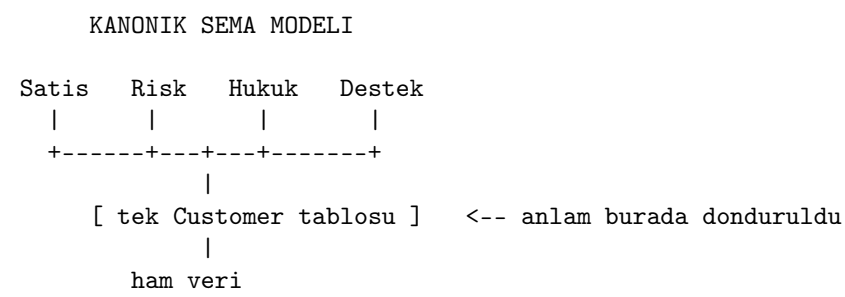
- **Satış** için müşteri, potansiyel gelirdir; henüz hesabı olmayan biri de müşteridir.
- **Risk** için müşteri, yükümlülük taşıyan taraftır; vefat etmiş biri, kredisi devam ettiği sürece hâlâ müşteridir.
- **Hukuk** için müşteri, imzalı sözleşmesi olan tüzel kişidir; şirketin sahibi müşteri değildir, şirket müşteridir.
- **Destek** için müşteri, arayan kişidir; sözleşme sahibinin eşi de müşteridir.

Bunlar aynı kelimenin farklı yorumları değil, **farklı nesnelerdir**. Her birimin işi, "müşteri"nin sınırlarını farklı bir yerden çizmeyi zorunlu kılmaktadır.

Aynı yapı savunma ve güvenlik alanında "hedef" kavramında da gözlenir. İstihbarat analisti için hedef, gözetlenen bir varlıktır; zamanla değişen konumu, bilinen ortakları ve güven skoru olan bir şeydir ve hedef olmak geri alınabilir bir statüdür. Dolandırıcılık analisti için hedef ise bir işlem örüntüsüdür; kişi olmak zorunda değildir, bir kart veya bir cihaz tanımlayıcısı olabilir ve hedef olmak bir skor eşiğinin aşılmasıdır.

Kanonik Şema ve Çöküşü

Klasik veri mühendisliği yaklaşımı, felsefi konumu itibarıyla erken Wittgenstein'cıdır: kurumda tek bir doğru `Customer` tablosu bulunur, tüm birimler ona uyar ve uymayan kendi tarafında çeviri yapar.



Yapısal sorun şudur: söz konusu tablo, taraflardan birinin dil oyununa göre yazılmıştır. Diğerleri kendi kavramlarını bu şemaya çevirmek zorunda kalır ve çeviride belirleyici olan unsur kaybolur. Risk birimi vefat etmiş müşteriyi, destek birimi sözleşme sahibinin eşini kaybeder.

Kaybın niteliği belirleyicidir. Kayıp gürültülü değil sessizdir: sistem çökmez, yalnızca yanlış çalışır. Bu nedenle teknik izleme araçlarıyla saptanamaz. Gözlemlenebilir sonucu, birimlerin kendi kayıt dışı tablolarını tutmaya başlamasıdır.

Ara Tespit: Çevirinin Kaçınılmazlığı

Buradan çıkan ve her iki cevabın da örtük olarak kabul ettiği ilke şudur:

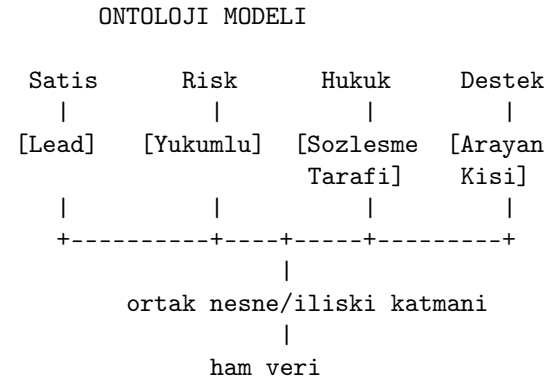
> Çeviri yok edilemez; yalnızca nerede yapılacağı seçilebilir.

Kötü tasarımda çeviri her yerdedir ve görünmezdir. İyi tasarımda çeviri sınırlı sayıda noktada toplanmıştır ve bilinçlidir. İncelenen iki yaklaşım, bu noktaların nerede olacağı konusunda ayrışmaktadır.

Birinci Cevap: Ontoloji

Yapı

Ontoloji yaklaşımı kanonik şemayı tersine çevirir:



İlke şudur: ham veri tektir, anlam katmanı çoğuldur. Her topluluk kendi nesne tiplerini, özelliklerini ve ilişkilerini tanımlar; bu tanımlar aynı kayıtların üzerine oturur ancak farklı kesitler alır.

Üreticinin kendi tarifiyle ontoloji, bir veri kataloğu veya şema tasarımı çözümü değil kurumun *operasyonel anlam katmanıdır*: nesneler, özellikler ve bağlantılardan oluşan semantik unsurlar ile bunlar üzerinde çalışan eylem ve fonksiyonlardan oluşan kinetik unsurları birlikte barındırır.

Ürünün çekirdek fikri, analistin zihinsel modelini yazılımda doğrudan temsil etmektir. Bir istihbarat analisti "şu numara şu kişiye ait, o kişi şu araca binmiş, o araç şu adreste görülmüş" biçiminde, nesneler ve ilişkiler üzerinden düşünür; oysa veri birbirinden kopuk veritabanlarında tablolar hâlinde durur. Ontoloji katmanı bu iki temsil arasındaki mesafeyi kapatır ve kullanıcının sorgu dili yerine ilişki grafiği üzerinde gezinmesini mümkün kılar.

Mimari Kısıtlar

Ürünün üç mimari özelliği, çalışmanın karşılaştırma bölümü açısından belirleyicidir.

Kapsam zorunluluğu. Veri yerinde sorgulanmaz; işleme hatlarından geçirilerek platforma alınır ve ontoloji nesnelerine dönüştürülür. Bu, kısmi uygulamayı zorlaştırır: ontolojinin değeri bağlantı kurabilmesinden geldiği için, kapsam dışında kalan her alan aynı zamanda değer kaybıdır.

Ontoloji-alan eşlemesi. Bir ontoloji, platformdaki bir çalışma alanıyla birebir eşleşir. Bu, ilerleyen bölümde ele alınacağı üzere, bir sınır kavramının tümüyle yok olmadığını ancak farklı bir ölçekte konumlandığını göstermektedir.

Ontolojiler arası bağ yasağı. Farklı ontolojilerdeki varlıklar birbirine bağlanamaz. Bu kısıt, çoklu ontolojiyi teknik olarak mümkün kılmakla birlikte pratikte cazibesiz hâle getirir, zira ürünün temel vaadi olan bağlantılanabilirlik tam da bu sınırda kesilir.

Ontoloji Uygulamalı Soyutlamadır

Ontoloji kurmak uygulamalı bir soyutlama işlemidir. Yüzeydeki değişken ayrıntı — hangi tablo, hangi sütun, hangi format — atılır; altındaki yapı tutulur.

Kavramsal soyutlamalardan farkı sınanabilirliğidir. Kâğıt üzerinde kalan bir kavramsal iskeletin yanlışlığı uzun süre görünmez kalabilir. Ontoloji ise çalışan bir şemaya dönüşür ve yanlış kurulduğunda sistem işlevsizleşir. Bu anlamda ontoloji, gerçeklikle çarpışarak sertleşmiş bir metafiziktir.

Bu özellik, çalışmanın Ontology'ye yönelttiği eleştirilerden bağımsız olarak, yaklaşımın kayda değer bir epistemolojik erdemidir.

Erken mi, Geç mi Wittgenstein?

Ontology'yi geç Wittgenstein'ın ürüne dönüşmüş hâli olarak sunan yaygın anlatı tam oturmamaktadır ve bu uyumsuzluk kavramın anlaşılmasını derinleştirmektedir.

Yapı erken Wittgenstein'a benzemektedir. Nesneler, özellikler ve ilişkilerden kurulu formal, sonlu ve makine tarafından işlenebilir bir model, doğrudan *Tractatus*'un dünya tasviridir.

Gerekçe geç Wittgenstein'a benzemektedir. Neden tek şema değil de çoğul şema sorusuna verilen cevap, anlamın topluluğa göreliliğine dayanır.

Yani ürün, çoğulculuğun *neden* gerektiğini geç Wittgenstein'dan, *nasıl* modelleneceğini erken Wittgenstein'dan almaktadır. Burada gerçek bir felsefi ironi bulunmaktadır: geç Wittgenstein'ın asıl iddiası anlamın tam olarak formalize edilemeyeceğidir; ontoloji ise tam olarak anlamı formalize etme girişimidir.

Dürüst formülasyon şudur: ontoloji, Wittgenstein'ın tespitini ciddiye alır ancak uyarısını göz ardı eder. Bu bir kusur değildir; mühendislik zaten böyle çalışır. Ancak "Wittgenstein'ın ürüne dönüşmüş hâli" ifadesi, olduğundan daha temiz bir düşünsel devamlılık ima etmektedir.

Olgusal Düzeltme

Yaygın anlatıda düzeltilmesi gereken bir olgusal hata bulunmaktadır: Palantir'in ilk ürününün adı Wittgenstein'a atıf değildir. Şirket adı Tolkien'den gelmektedir (*palantír*, gören taş); ilk ürün Palantir Government, sonrasında Gotham'dır.

Wittgenstein bağlantısı ürün adında değil, ontoloji kavramının kendisindedir ve o kelimenin kökü Aristoteles'e kadar gitmekte olup Wittgenstein'a özgü değildir.

Alex Karp'ın Frankfurt'ta tamamladığı doktora tezinin başlığı *Aggression in der Lebenswelt*'tir. Bu tezin "ontoloji" terimine başvurmayı ideolojik bir işlem olarak eleştirdiği yönündeki iddia ikincil kaynaklarda dolaşmaktadır; ancak birincil metin üzerinden doğrulanmamıştır ve bu çalışmada kanıt olarak kullanılmamaktadır.

Arşivlenecek doğru formülasyon şudur: ontoloji kavramının savunusu geç Wittgenstein'cı bir argümana dayanmakta, yapısı ise erken Wittgenstein'cı kalmaktadır.

İkinci Cevap: Domain-Driven Design

Bu bölüm, yaklaşımın yalnızca karşılaştırma için gerekli unsurlarını özetlemektedir. Ayrıntılı sunum için kaynakçadaki metinlere başvurulmalıdır.

Değer Önerisi ve Uygulanabilirlik Koşulu

Yaklaşımın tek cümlelik değer önerisi, domain bilgisini merkezîleştirmek ve korumaktır. Teknik detayların, veri modelinin, arayüz ihtiyaçlarının ve çerçeve alışkanlıklarının iş bilgisine sızıp onu bozması engellenir.

Yaklaşımın kendisi bir maliyet kalemidir; başlangıç yatırımı yüksektir ve ilerleme başlangıçta yavaş hissedilir. Bu nedenle uygulanabilirlik koşulu açıkça tanımlanmıştır: kolayca değiştirilebilen sistemlerde uygulanmaz, uzun ömürlü ve iş açısından kritik sistemler içindir. Belirleyici soru karmaşıklığın kaynağıdır — problem doğası gereği mi zordur, yoksa çözüm mü karmaşıklaştırılmaktadır?

Stratejik ve Taktiksel Ayrım

	Strategic Design	**Tactical Design**
Soru	Neyi, neden inşa ediyoruz?	Bunu nasıl modelleriz?
Araçlar	Domain, Subdomain, Bounded Context, Context Map	Entity, Value Object, Aggregate, Domain Service
Odak	İş ve organizasyon yapısı	Kod

Kritik bağımlılık şudur: taktiksel tasarım, stratejik tasarımın çizdiği sınırlar içinde anlam kazanır ve sıra tersine çevrilemez. Stratejik tasarımın kapsamı yalnızca yazılımla sınırlı değildir; ekipler arası sınırları da belirlediği için organizasyonu da tasarlar.

DDD Lite

Stratejik kısmın atlanıp yalnızca taktiksel araçlara odaklanıldığı uygulama biçimi literatürde DDD Lite olarak adlandırılmaktadır. Dışarıda bırakılan üç unsur şunlardır: ortak dil inşası, bağlamsal sınırlar ve bağlamlar arası ilişki haritası. Bunların yerine sırasıyla teknik terminoloji, kod içi klasörleme yapısı ve tasarlanmamış entegrasyonlar geçer.

Bu kavram, çalışmanın karşılaştırma bölümünde merkezî bir işlev görecektir.

Bounded Context: Dilsel Sınır

Bounded Context, bir domain modelinin yaşadığı sınırdır ve sınır içinde konuşulan dil Ubiquitous Language'dir. Tanımın en işlevsel sonucu bir teşhis kriteri sunmasıdır:

> Bir kavramın birden fazla anlama geldiği düşünülüyorsa, bağlam fazla büyümüştür; aslında birden fazla bağlamın içinde bulunulmaktadır.

Kanonik örnek, bankacılık domain'indeki `Account` kavramıdır. Vadesiz hesap bağlamında `Account`; para yatırma, çekme, bakiye takibi ve günlük işlem limitleri davranışlarına sahiptir. Vadeli hesap bağlamında ise faiz hesaplama, vadeye bağlı kurallar ve birikim davranışlarıyla tanımlanır. Her iki bağlamda da nesnenin adı `Account` olabilir; anlam farkını sınır belirlemektedir.

Sınırın kapsamı yalnızca domain katmanı değildir: kalıcılık şeması, uygulama servisleri, arayüz katmanı ve dışa açılan uç noktalar da bu sınırın içindedir. Sahiplik ilkesi "tek bağlam, tek dil, tek takım" biçiminde formüle edilir.

Ubiquitous Language

Ubiquitous Language, bir işi anlatmak için kullanılan kelimelerin, o işin geçtiği her yerde aynı tutulması disiplinidir. Gerekçesi çeviri maliyetidir: bir kavram aktarım zinciri boyunca farklı isimler aldığında, her aktarımda taşıdığı bir unsur sessizce düşer.

Yaklaşımın üç olumsuz tanımı belirleyicidir. Ubiquitous Language *yalnızca iş tarafının dili değildir*, zira iş tarafı kendi içinde tutarlı olmak zorunda kalmamıştır. *Sektör terminolojisi değildir*, zira genel terimler kurumu ayırt eden farklılığı adsız bırakır. *Domain uzmanının tek başına dili de değildir*, zira işi en iyi bilen kişi onu anlatmakta en iyi kişi değildir; uzun süredir bir işi yapan kişi kuralları refleks olarak uygular ve sorulduğunda formüle edemez.

Dil, domain uzmanı ile geliştiricinin temas noktasında doğar: birincisi bilgiyi, ikincisi kesinlik talebini getirir.

Sürtünme sinyali

Epistemolojik olarak en değerli tespit şudur: bir kelimeyi kullanırken duyulan rahatsızlık genellikle kelime sorunu değil, anlayış sorununun göstergesidir. "Sipariş" kelimesi rahat kullanılırken "sepete eklendi ama ödenmedi, bu da sipariş midir?" sorusu karşısında duraksanıyorsa, kelimenin iki farklı şeyi birden taşıdığı ortaya çıkmıştır.

Ters durumda ise şu gözlenir: hiç rahatsızlık üretmeyen kelimelerin ortak özelliği hiçbir şey söylememeleridir — "kayıt", "bilgi", "işlem", "birim", "öğe". Bunlar hiçbir yere yerleştirilirken zorlanma yaratmaz, çünkü hiçbir şeyi ayırt etmezler. Sıfır sürtünme genellikle sıfır anlam göstergesidir.

Anemic Domain Model

Fowler'ın 2003'te adlandırdığı bu anti-pattern, veri taşıyıp davranış taşımayan modelleri tanımlar.

Kavramı somutlaştırmak için şu karşıtlık kullanılabilir. Bir varlık iki biçimde temsil edilebilir: birincisinde varlık kendi kilidine sahip bir kasadır ve talep edilen işlemi gerçekleştirmeden önce koşulları kendisi denetler; ikincisinde varlık üzerinde bir değer yazılı bir kâğıttır ve kuralları bilen ayrı birimler bulunur. Anemic Domain Model ikinci temsildir.

Asıl maliyet, kuralların model dışına dağılmasının iki sonucudur. Birincisi, yeni eklenen bir yolun mevcut kuralı farkında olmadan atlayabilmesidir; model itiraz etmediği için hata sessizce geçer. İkincisi, bir kural değiştiğinde onu bilen tüm birimlerin tek tek bulunup düzeltilmesi gereğidir; biri atlandığında kural sistemde iki farklı biçimde işlemeye başlar.

Nüans önemlidir: anemik model her zaman hata değildir ve korunacak bir kuralın bulunmadığı basit alanlarda bilinçli bir tercih olabilir. Asıl sorun, anemik model kullanıldığının fark edilmemesidir.

Context Map ve İlişki Pattern'leri

Context Map, bağlamların ve aralarındaki ilişkilerin haritasıdır. Belirleyici ilkesi bugünü çizmesidir: hayal edilen mimariyi değil mevcut durumu gösterir.

Harita üç katmanı aynı anda temsil eder: modeller arasındaki ilişki, dillerin sınırda nasıl uyumlanacağı ve ekipler arasındaki organizasyonel dinamik. Üçüncü katman, başka hiçbir diyagram türünde bulunmaz ve bu çalışmanın karşılaştırma bölümünün dayanak noktasıdır.

Dokuz pattern

Pattern	**Özü**	**Bedeli**
Partnership	Ortak kader, ortak sürüm	Sürekli koordinasyon
Shared Kernel	Modelin bir parçası ortak	Çok yakın bağımlılık
Customer/Supplier	Talepler planlamaya girer	Pazarlık maliyeti
Conformist	Üst model aynen benimsenir	Kontrolün devri
Anticorruption Layer	Sınırda çeviri katmanı	Katmanı yaşatma
Open Host Service	Çok tüketiciye açık protokol	Protokol taahhüdü
Published Language	Belgeli ortak yayın dili	Sürüm yönetimi
Separate Ways	Bilinçli olarak bağlanmama	İşlevin yinelenmesi
Big Ball of Mud	Karmaşayı karantinaya alma	Kurtarmadan vazgeçme

Bu dokuz pattern, esasen tek bir soruya verilen farklı cevaplardır: iki topluluk anlaşamadığında kimin dili geçerli olacaktır?

İki uyarı, karşılaştırma bölümü açısından özellikle önemlidir.

İlan edilmemiş Shared Kernel. Bir bağlamın veritabanı diğerinin tarafına çoğaltıldığında, resmen kararlaştırılmadan bir Shared Kernel yaratılmış olur. Çoğaltma, beklenenin aksine özerklik getirmez; iki tarafı aynı şemaya bağlar ve bu bağ hiç müzakere edilmemiştir.

Bilinçsiz Conformist. Conformist olmak bilinçli yapıldığında makul bir ödünleşimdir; asıl sorun farkında olmadan Conformist olmaktır. Dış bir arayüzün veri taşıyıcıları domain modelin içine kadar sızmışsa, karar verilmeden verilmiş demektir.

Son olarak, roller bağlama değil ilişkiye aittir: aynı bağlam bir ilişkide üst konumda, diğerinde alt konumda bulunabilir.

Sınırda Çeviri

Context Map, bağlamlar arasındaki ilişkiyi etiketlerle gösterir. Bir sonraki soru, sınırı geçen verinin karşı tarafa hangi biçimde girdiğidir. Ürkmez bu soruyu, bir e-ticaret modelinde kimlik bilgisini yöneten Identity bağlamı ile sipariş süreçlerini yöneten Order bağlamı arasındaki hat üzerinden işler (Ürkmez, 2026e).

Üst taraf: Open Host Service ve Published Language

Üst taraftaki bağlamın birden çok tüketicisi vardır ve her tüketici için ayrı bir entegrasyon tasarlamak sürdürülemez. Open Host Service bu soruna tek bir açık protokolle cevap verir. Published Language ise bu protokolde kullanılan, belgelenmiş ve kararlı temsil biçimidir. Üst taraf iki tür kaynak açabilir: doğrudan sorgu adresleri ve önemli olayların yayınlandığı bir bildirim akışı. Tüketici bu akışı kendi hızında okur; aynı bildirimi iki kez işlememek tüketicinin sorumluluğudur.

Bir bağlamın dışarıya yayınladığı Domain Event'ler de Published Language'in parçasıdır. Event'ler, bağlamın kamuya açık sözlüğü işlevini görür.

Alt taraf: Anticorruption Layer'ın üç rolü

ACL tek bir sınıf değil, üç rolden oluşan bir zincirdir:

- **Domain Service**, yerel modelin kapısıdır. Model "bana şu müşteriyi ver" der ve uzak sistemin varlığından habersizdir.
- **Adapter**, uzak sistemle konuşan istemcidir. Üst tarafın protokolüne gider ve temsili alır.

- **Translator**, dönen temsili yerel modelin kavramına çevirir; ortaya yerel bir Value Object çıkar.

Bu zincirin hiçbir halkasında uzak bağlamın dili yerel modelin içine girmez.

Translation Map ve asgari state

Translator'ın işi bir eşleme tablosuyla tarif edilir. Identity bağlamının kullanıcı temsilinde kiracı kimliği, kullanıcı adı, ad, soyad, e-posta ve rol alanları bulunduğu varsayılsın. Order bağlamının ihtiyacı yalnızca üç kavramdır: müşterinin kimliği, adı ve e-posta adresi. Ad ile soyad birleşip tek bir ad alanına, kullanıcı adı kimliğe dönüşür; geri kalan alanlar eşlenmeden dışarıda kalır.

İlke şudur: uzak modelden alınan state, yerel modelin ihtiyaç duyduğu asgari alanlardan ibarettir. Ürkmez bu ilkenin ihlalini **hibritleşme** olarak adlandırır (Ürkmez, 2026e). "Nasılsa lazım olur" gerekçesiyle eklenen her alan, yerel `Customer` kavramını uzak `User` kavramının melez bir kopyasına yaklaştırır. Çevirinin giderek karmaşıklaşması ve çevrilen nesnenin karşı modeldekine benzemeye başlaması, yabancı bağlamdan fazla şey alındığının erken uyarısıdır.

Bu tespit, kanonik şema bölümündeki çöküşün ters yönden yeniden üretilmesidir. Kanonik şemada tüm birimler tek bir kavrama sıkıştırılır; hibritleşmede ise bir birimin kavramı, farkında olunmadan bir başkasının kavramına doğru büyür. İki durumda da sonuç, sınırın dilsel işlevini yitirmesidir.

Senkron ve asenkron entegrasyon

Adapter, en basit hâliyle uzak sisteme senkron bir istek atar ve cevabı bekler. Kurulumu kolaydır, ancak bedeli çalışma anında ödenir: üst taraf cevap veremediğinde alt tarafın akışı da durur. Asenkron yaklaşımda ise alt taraf, üst tarafın yayınladığı bildirimleri kendi hızında işler ve ihtiyaç duyduğu asgari state'i yerelde güncel tutar. Böylece çevresindeki sistemlerin o an ayakta olup olmamasından etkilenmez. Ürkmez, ilk sürümde senkron başlayıp sonradan asenkrona geçmeyi meşru bir yol olarak görür; belirleyici olan, geçiş kararının bilinçli verilmesidir (Ürkmez, 2026e).

Ara durumun adlandırılması

Asenkron entegrasyon, iki bağlamın birbiriyle tutarlı olmadığı bir ara an yaratır. Bir sipariş oluşturulduğunda stok rezervasyonu henüz yapılmamış olabilir. Bu ara an bir hata değil, modellenmesi gereken geçerli bir durumdur. Order bağlamı rezervasyonun durumunu açıkça tutar: `REQUESTED`, `READY`, `FAILED`. Kullanıcı hiçbir anda belirsizlikte kalmaz, çünkü sistemin her ara anı adı olan bir duruma karşılık gelir.

Bu çözüm, kavramsal zemindeki *sessiz hata* tanımına doğrudan bir cevaptır. Sessiz hata, sistemin yanlış çalıştığı ancak bunu bildirmediği durumdur. Ara

durumun adlandırılması, aynı belirsizliği görünür ve sorgulanabilir bir nesneye dönüştürür.

Taktiksel Yapı Taşları

Stratejik tasarım sınırları çizer; taktiksel tasarım bu sınırların içini kodla doldurur. Bu alt bölüm, karşılaştırma için gerekli olan beş yapı taşını özetlemektedir.

Entity: kimlik ve zaman

Bir kavramın Entity olarak modellenmesinin tek gerekçesi, tekilliğinin önemsenmesidir (Ürkmez, 2026f). Bir sipariş oluşur, ödenir, kargoya verilir, belki iptal edilir; durumu ve adresi değişir ama aynı sipariş olarak kalır. Entity'yi tanımlayan taşıdığı veri değil, kimliği ve zaman içindeki değişimidir. İki siparişin tutarı, adresi ve kalemleri aynı olabilir; yine de aynı sipariş değildir.

Kimliğin kaynağı için dört strateji vardır: kullanıcının girdiği değer, uygulamanın ürettiği değer (örneğin UUID), veritabanının ürettiği değer ve başka bir bağlamın verdiği değer. Kullanıcı girdisi okunaklıdır ancak hataya açıktır; bu nedenle kullanıcının girdiği kod çoğunlukla aranabilir bir özellik olarak tutulur ve kimlik arka planda ayrıca üretilir. Veritabanının ürettiği kimlik ise ancak kayıt anında doğar ve nesne bir süre kimliksiz yaşar. Bu durum, kimliğe nesne oluşturulduğu anda ihtiyaç duyan Domain Event'lerle çatışır: event yayınlandığında içinde geçerli bir kimlik bulunmaz. Bu yüzden event yayınlayan Entity'lerde kimliğin erken atanması tercih edilir.

Kimlik, çıplak bir `string` yerine kendine ait bir tipte tutulur. Böylece `OrderId` bekleyen bir metoda `ProductId` verilmesi derleme anında engellenir. İki kimlik aynı ham tipte tutulduğunda ise bu hata derleyici tarafından görülmez ve sistem yanlış veriyle sessizce çalışmaya devam eder (Ürkmez, 2026f).

Entity'nin davranışı dilden keşfedilir. "Sipariş iptal edilebilir" cümlesi ilk bakışta tamdır. Domain uzmanıyla konuşma sürdürüldüğünde cümle değişir: "Müşteri, siparişini kargoya verilene kadar iptal edebilir; kargoya verilmiş siparişler için iade süreci başlatılır." İlk cümleden bir kural çıkmaz; ikinci cümle metodun ilk satırını kendisi yazar. `order.Status = Cancelled` biçimindeki bir atama, kuralın her çağıran tarafından tekrarlanmasını gerektirir; `order.Cancel()` ise kuralı verinin yanında tutar. Evans bu tür arayüzlere niyetini gizlemeyen arayüz (*Intention-Revealing Interface*) adını verir (Evans, 2003).

Value Object: değer ve değişmezlik

Value Object, Entity'nin karşıt kutbudur: bir nesnenin yalnızca taşıdığı öznitelikler yeterliyse, o nesne bir Value Object'tir (Ürkmez, 2026g). Value bir şeyi ölçer, niceler ya da tanımlar; kendisi o şey değildir. Bir tutar tek başına bir varlık değil, bir siparişin parasal değeridir.

Ürkmez, Value Object'i tanımlayan altı karakteristik sayar:

1. **Bütünlük.** Alanlar tek başına anlamsızdır, birlikte bir değer oluşturur. 100 sayısı ile `TRY` kodu ayrı ayrı bir şey söylemez; birlikte bir tutardır.
2. **Değişmezlik.** Oluşturulduktan sonra değişmez.
3. **Yer değiştirebilirlik.** Değer değiştiğinde eskisi güncellenmez, yerine yenisi konur.
4. **Değer eşitliği.** Aynı alanları taşıyan iki nesne eşittir.
5. **Yan etkisiz davranış.** Metotları kendi durumunu değiştirmez, yeni bir değer döndürür.
6. **Entity'den bağımsızlık.** Bir Value'nun metodu parametre olarak Entity almaz, yalnızca başka Value'larla çalışır.

Value Object'in hedef aldığı alışkanlığın literatürdeki adı *primitive obsession*'dır (Fowler, 1999). Tip anlam taşımadığında, o anlamı korumak kodun her yerine dağılır. Bir tutar `decimal` ve `string` olarak iki ayrı alanda tutulduğunda, para birimi denetimi her kullanımda yeniden yazılmak zorundadır; `Price` tipi ise farklı para birimlerinin toplanmasını kendisi reddeder.

Ters yöndeki risk de adlandırılmıştır: gerçekten yalın ve başka hiçbir alanla ilişkisi olmayan bir değeri tipe sarmalamak yarardan çok karmaşıklık getirir. Önceden bilinen sınırlı bir kümeden değer alan tipler (sipariş durumu, para birimi) ise Standard Type adıyla ayrı bir Value Object ailesi oluşturur.

Value Object'in bu çalışmadaki önemi, sınırda çeviriyle doğrudan bağından gelir. Translation Map'in ürettiği üç alanlık `Customer`, uzak bağlamın zengin temsilini yerel ihtiyaca indiren bir Value Object'tir. ACL'nin çıktısı, taktiksel düzeyde bir Value Object olarak somutlaşır.

Domain Service: sahibi olmayan davranış

Evans'ın tanımına göre, domain'deki önemli bir süreç ya da dönüşüm doğal olarak bir Entity'nin veya Value Object'in sorumluluğu değilse, modele ayrı bir arayüz olarak eklenir; arayüz modelin diliyle adlandırılır ve durumsuz tutulur (Evans, 2003). Ürkmez bu tanımı işlemsel bir ölçüte çevirir: bir kural tek bir Aggregate'in içindeki bilgiyle çalıştırılamıyorsa Domain Service gerekir (Ürkmez, 2026h).

Örnek, bir siparişin ödenecek tutarıdır. Kural, siparişin kalemlerini, satıcının o an aktif kampanyasını ve müşterinin kuponunu aynı anda görmeyi gerektirir; ayrıca toplam indirim sepet tutarının yarısını geçemez. Bu hesap uygulama servisinde yazıldığında kampanya ve kupon birer veri paketi gibi okunur ve kendi kurallarını işletmez; sonuç Anemic Domain Model'dir. `Order` sınıfının içine yazıldığında ise sipariş iki yabancı kavramı tanımak zorunda kalır. Doğru yer, adı domain dilinden gelen bir `OrderPricingService`'tir. Servis, "siparişi işle" gibi

teknik bir etiketle değil, domain uzmanının kurduğu "sipariş tutarını hesaplıyoruz" cümlesinden adlandırılır.

Ürkmez'in vurguladığı risk aracın kendisindedir: yerini bulamayan her kuralı servise atmak modeli boşaltır ve Domain-Driven Design'ın kaçındığı anemik yapıyı yeniden üretir. Önerilen sezgi, Ubiquitous Language bölümündeki sürtünme sinyalinin taktiksel karşılığıdır. Bir davranışı bir nesneye yerleştirirken zorlanılıyorsa, davranış o nesneye ait olmayabilir; servisin içindeki kod tek bir Entity'nin alanlarıyla oynuyorsa, davranışın adresi o Entity'dir.

Ürkmez ayrıca karar veren servis ile çeviri yapan servisi ayırır. Bir adresin bir kargo firmasının hizmet alanında olup olmadığına karar veren servis bir Domain Service'tir. Kargo firmasının durum kodunu yerel durum tipine dönüştüren servis ise bir dönüşüm servisidir (*transformation service*); arayüzü domain katmanında dursa bile bir karar vermez. Arayüzün nerede durduğu bağımlılığın yönünü belirler; operasyonun ne yaptığı ise domain'e ait olup olmadığını.

Aggregate: tutarlılığın sınırı

Aggregate kelimesi bir yığın çağrıştırır; anlattığı şey ise bir sınırdır. Sorulması gereken soru "hangi nesneler bir arada durmalı" değil, "hangi kurallar her an, aynı anda doğru kalmalı"dır (Ürkmez, 2026i). Aggregate bir nesne grafiği ya da sahiplik şeması değildir; bir tutarlılık sınırıdır ve tek bir transaction içinde bütün kurallarıyla tutarlı hâle gelebilmelidir.

Ürkmez'in örneği, "bir siparişin kalemleri, ödemesi, kargosu ve durum geçmişi vardır" cümlelerinden türetilmiş dev bir `Order`'dır. Model domain uzmanının cümleleriyle birebir örtüşür. Ancak üretimde, birbiriyle ilgisiz iki işlem — bir kalemin iadesi ile bir kolinin kargoya verilmesi — aynı nesne üzerinde çakışır. Kök neden bir **sahte invariant**'tır: gerçek bir iş kuralı olmayan, geliştiricinin kendi koyduğu bir kısıt.

Sahte invariant'ı tanımanın yolu dilseldir: kural, domain uzmanına cümle olarak söylenir. "Sipariş iade edilirken kargo bilgisinin aynı anda değişmemesi gerekiyor" cümlesini duyan bir operasyon yöneticisi bu kuralı tanımaz, çünkü böyle bir kural yoktur. Gerçek invariant ise tanınır: "Siparişin ödenecek tutarı, kalemlerinin toplamıyla tutarlı olmak zorundadır." Bu nedenle `Order` ile kalemleri aynı sınırın içindedir; ödeme, kargo ve iade ise kendi kimlikleri olan ayrı Aggregate'lerdir.

Ürkmez dört kural sayar:

1. Gerçek invariant'lar sınırın içinde modellenir.
2. Aggregate küçük tasarlanır. Büyük Aggregate'ler tek bir alanı okumak için onlarca nesneyi belleğe yükler.
3. Diğer Aggregate'lere nesneyle değil, kimlikle referans verilir. Böylece bir transaction içinde iki Aggregate'in birden değiştirilmesi kodda zorlaşır.
4. Sınırın dışında eventual consistency kullanılır.

Bir kuralın transactional mı eventual mı tutulacağına dair Evans'ın ayracı da dilseldir: veriyi tutarlı hâle getirmek, işlemi yapan kullanıcının işi midir? Sipariş tutarının kalemlerle tutarlı olması müşterinin işidir, çünkü müşteri o tutar üzerinden karar verir; bu kural transactional tutulur. Stok rezervasyonu ise sistemin işidir ve eventual tutulabilir. Gecikmeye ne kadar tahammül edildiği de domain uzmanına sorulur; cevap çoğunlukla geliştiricinin beklediğinden cömerttir, çünkü işin kendisinde zaten gecikmeler vardır.

Domain Event: sınırın dışına taşan olay

Domain Event, bir Aggregate'te gerçekleşmiş ve geriye dönük değiştirilemeyen bir olayın kaydıdır; Aggregate'ler arasındaki eventual consistency'nin köprüsüdür (Ürkmez, 2026j). Domain uzmanının "... onaylandığında", "... tükenirse", "bana haber ver, eğer ..." kalıplarındaki cümleleri bir event sinyalidir. Event, kendisini üreten komuttan türetilir ve geçmiş zamanla adlandırılır: `Confirm()` komutu `OrderConfirmed` olayını üretir. Adın kendisi Ubiquitous Language'e bağlıdır; o domain'de siparişler "tamamlanmıyor" da "onaylanıyorsa", event'in adı da buna göre konur.

Event, transaction başarıyla tamamlandıktan sonra yayınlanır; transaction geri alınırsa gerçekleşmiş bir şey yoktur ve event de yayınlanmamalıdır. Bağlam dışına çıkacak event'ler, modelle aynı transaction içinde ayrı bir tabloya yazılır ve ayrı bir süreç tarafından mesajlaşma altyapısına iletilir; bu düzenin adı Outbox'tır. Mesajlaşma altyapıları çoğunlukla "en az bir kez" teslim garantisi verdiğinden, aynı event bir tüketiciye iki kez ulaşabilir. Tüketici bu nedenle idempotent tasarlanır: aynı event ikinci kez geldiğinde sonuç değişmez.

İki olumsuz tanım belirleyicidir. Domain Event bir bildirim değildir; bildirim, event'e verilen tepkilerden biridir. Her durum değişikliği de bir event değildir; event, domain uzmanının bilmek istediği yerde devreye girer.

Taktiksel Kararların Dilsel Dayanağı

Beş yapı taşının ortak bir özelliği vardır: her birinde belirleyici karar, domain uzmanına söylenen bir cümlenin sınanmasıyla verilir.

Yapı taşı	**Karar**	**Dilsel sınama**
Entity	Hangi davranış hangi kuralı taşır	Gereksinim cümlesi uzmanla yeniden yazılıyor mu
Value Object	Hangi alanlar tek bir değer oluşturur	Alanlar tek başına bir şey söylüyor mu
Domain Service	Davranışın bir sahibi var mı	Servisin adı uzmanın kullandığı bir eylem mi
Aggregate	Sınır nereden geçer	Invariant cümlesini uzman tanıyor mu
Domain Event	Hangi olay yayınlanır	Uzman bu olaydan haberdar olmak istiyor mu

Bu tablo, felsefi çerçevedeki kural takibi tespitinin mühendislikteki karşılığıdır. "Sipariş iptal edilebilir" bir kuraldır, ancak kendi uygulamasını belirlemez. Kargodaki bir siparişin kapsam içinde olup olmadığı, kuralın metninden değil, onu uygulayan topluluğun pratiğinden öğrenilir. Domain-Driven Design formalizasyondan vazgeçmez; formalizasyonun her adımını, pratiğin sahibi olan kişiye geri götürülen bir sınamaya bağlar. Böylece Wittgenstein'ın itirazı ortadan kalkmaz, ancak her taktiksel kararda yeniden sorulabilir hâlde kalır.

Karşılaştırma

Kavram Eşlemesi

DDD	Ontology	Durum
Entity	Object Type	Yakın
Value Object	Property	Kısmi
Domain Service	Function	Yakın
Aggregate komutu	Action	Kısmi
Domain Event	—	Belirsiz
Ubiquitous Language	Nesne tipi isimlendirmesi	Kısmi
ACL / Translation Map	Veri hattı dönüşümleri	Farklı konumda
Bounded Context	Ontoloji-alan eşlemesi	Farklı ölçekte
Aggregate	—	Karşılığı yok
Context Map	—	Karşılığı yok
İlişki pattern'leri	—	Karşılığı yok
Subdomain tipolojisi	—	Karşılığı yok

Tablodaki en önemli satır Bounded Context satırıdır ve dikkatli okunmalıdır. Ontology'de bir sınır kavramının bulunmadığını söylemek yanlış olur: ontolojiler çalışma alanlarıyla eşleşmekte ve aralarında bağ kurulamamaktadır. Doğru tespit, sınırın *yok olması* değil *ölçek değiştirmesidir*. Domain-Driven Design'da sınır kurumun içinden geçer ve topluluklar arasına konur; Ontology'de sınır kurumun etrafını çevirir ve dış dünyaya karşı konur.

Taktiksel satırlar ayrıca açıklanmalıdır. Value Object ile Property arasındaki eşleşme kısmidir: bir Property bir değer taşır, ancak Value Object'in değer önerisi değeri değil, değere bağlı kuralı ve yan etkisiz davranışı taşımasıdır. Action ile Domain Event arasında ilk bakışta kurulabilecek eşleşme ise yanıltıcıdır. Action, bir kullanıcının nesneler üzerinde yaptığı değişikliği tanımlar; Domain-Driven Design açısından bir event'ten çok Aggregate üzerindeki komut metoduna yakındır. Gerçekleşmiş olayın geçmiş zamanla adlandırılmış bir kaydı olarak diğer bağlamlara yayılması ise bu çalışmanın incelediği kaynaklarda Domain Event'le eşdeğer bir kavram olarak tanımlanmamıştır; bu nedenle satır "belirsiz" olarak işaretlenmiştir.

Bu ayrımların sonucu şudur: karşılığı bulunmayan kalemlerin Aggregate dışındakilerin tamamı stratejik taraftadır. Aggregate taktiksel bir yapı taşıdır; ancak işlevi küçük ölçekte bir sınır çizmektir. Dolayısıyla karşılığı bulunmayan kalemlerin ortak paydası stratejik olmaları değil, *sınır çizmeleri ve sınırlar arası ilişkiyi adlandırmalarıdır*. Ontology, Domain-Driven Design'ın nesne, özellik ve fonksiyon düzeyindeki taktiksel araçlarıyla büyük ölçüde örtüşmekte; büyük ölçekte Bounded Context ve Context Map, küçük ölçekte Aggregate olmak üzere sınır çizen araçlarıyla örtüşmemektedir.

Birinci Ayrım: Gücün Görünürlüğü

Context Map'in dokuz pattern'i bir müzakere spektrumu oluşturur:

```
      KIMIN DILI GECERLI?

Partnership       -> ikisi birlikte karar verir
Shared Kernel     -> ortak parca, ortak sahiplik
Customer/Supplier -> ust taraf karar verir, alt taraf pazarlik eder
Conformist        -> ust tarafin dili aynen kabul edilir
ACL               -> alt taraf kendi dilini korur, sinirda cevirir
OHS/PL            -> ust taraf herkese tek bir dil yayinlar
Separate Ways     -> hic konusmama karari
Big Ball of Mud   -> dil kaybedilmis, karantina
```

Ontology'de bu spektrum bulunmaz; tek bir konfigürasyon vardır ve tüm birimler ona bağlanır. Domain-Driven Design terimleriyle ifade edilirse Ontology, kurum çapında ilan edilmiş bir Shared Kernel ve o kernel'e karşı herkesin Conformist olmasıdır.

İki uyarı burada birleşmektedir. Shared Kernel için kural, paylaşılan parçanın olabildiğince küçük tutulmasıdır; Ontology ise paylaşılan parçayı bilinçli olarak büyütür, zira değeri kapsamından gelmektedir. Conformist için kural, kararın bilinçli alınmasıdır; Ontology ise bu kararı bir konfigürasyon işlemine dönüştürerek görünmezleştirir.

Sonuç şudur: Ontology, gücün nerede olduğu sorusunu ortadan kaldırmaz; cevabı altyapıya gömer. Hangi topluluğun kavram dünyasının kanonik hâle geldiği bir tasarım kararıdır, ancak hiçbir yerde bu adla kaydedilmez.

İkinci Ayrım: Seçicilik

Core, Supporting ve Generic ayrımı bir sınıflandırma egzersizi değil bir yatırım kararıdır: amaç her alanı iyileştirmek değil, değer üreten bilgiyi korumaktır.

Ontology'de bu ayrım bulunmaz. Ontology kapsayıcıdır; değeri birbirinden uzak veriler arasında bağlantı kurabilmesinden gelir ve bu değer ancak yüksek kapsama oranıyla gerçekleşir.

İki farklı bahis söz konusudur:

- **Domain-Driven Design'ın bahsi**: Karmaşıklık her yerde eşit değildir; çaba Core'a yoğunlaştırılmalıdır.
- **Ontology'nin bahsi**: Asıl değer bağlantıdadır; bu nedenle her şey kapsam içinde olmalıdır.

Pratik sonucu, Ontology'nin kısmi uygulanmaya elverişsiz olmasıdır. Domain-Driven Design ise seçici uygulanabilir; yalnızca Core'da uygulanıp geri kalanın basit tutulması meşru bir stratejidir. Taktiksel literatür bu seçiciliği tek tek yapı taşları düzeyinde de tekrarlar: korunacak ciddi bir iş kuralı bulunmayan alanlarda Entity yerine CRUD tabanlı bir çözüm hem daha ucuz hem daha doğrudur (Ürkmez, 2026f).

Üçüncü Ayrım: Anlamın Yaşadığı Yer

```
DDD'DE ANLAM NEREDE              ONTOLOGY'DE ANLAM NEREDE

kod  ->  tip sistemi             konfigurasyon -> arayuz
     |                                        |
derleme zamani kontrol           calisma zamani yorumlama
     |                                        |
surum kontrolu, inceleme         yonetim paneli, yetki
     |                                        |
sahibi: gelistirici ekibi        sahibi: platform/analist
```

Domain-Driven Design'da bir kavramın değiştirilmesi bir kod değişikliğidir: tartışılır, incelenir, geçmişi kayıt altına alınır ve tip sistemi tarafından denetlenir. Kimliğin ve değerlerin ayrı tiplerde tutulması (`OrderId`, `Price`) bu denetimi derleme zamanına taşır. Ontology'de bir kavramın değiştirilmesi bir konfigürasyon değişikliğidir: hızlıdır ve teknik olmayan personel tarafından yapılabilir, ancak daha az sürtünmeli ve daha az izlenebilirdir.

Bu fark, "anlamı kim korumaktadır?" sorusunun cevabını değiştirir. Domain-Driven Design'ın ana metaforu duvardır — sınır, koruma, çeviri. Ontology'nin ana metaforu katmandır — serilen, her yere uzanan. Birincisinin vaadi koruma, ikincisininki erişimdir.

Dördüncü Ayrım: Harita ile İkiz

Context Map'in tanımlayıcı ilkesi bugünü çizmesidir; bu nedenle bozulmuş bölgeler bile haritaya dâhil edilir. Ontology ise kendisini kurumun dijital ikizi olarak tanımlar.

İkiz, harita değildir. Harita ölçekli bir sadeleştirmedir ve sadeleştirdiğini beyan eder; ikiz eksiksizlik iddiası taşır.

Bu ayrımın yalnızca retorik olmadığı, mimari kısıtlardan çıkarılabilir. Ontology'nin kapsam zorunluluğu — kısmi uygulamanın değer üretmemesi — fiilî bir birebirlik baskısı yaratmaktadır. Yani eksiksizlik iddiası bir slogandan ibaret değil, ürünün değer önerisinin yapısal sonucudur.

Bu tespit, birinci bölümdeki felsefi gerilimi yeniden üretmektedir:

```
Tractatus       : dunya <-> dil, birebir   ~ dijital ikiz
Sorusturmalar   : ortusen kismi pratikler  ~ context map
```

Domain-Driven Design, felsefi bir iddiada bulunmaksızın geç Wittgenstein tarafında konumlanmaktadır. Ontology ise retorik olarak geç Wittgenstein'a yaslanırken pratikte erken Wittgenstein projesini yürütmektedir.

Beşinci Ayrım: Çevirinin Konumu

Kanonik şema bölümünün ara tespiti, çevirinin yok edilemeyeceği, yalnızca yerinin seçilebileceğiydi. İki yaklaşım bu seçimi farklı yapar.

Ontology'de çeviri, ham verinin işleme hatlarından geçirilip nesnelere dönüştürüldüğü noktada, yani kaynak sistem ile ortak katman arasında yapılır. Çeviri bir kez yapılır ve tek yönlüdür: her kaynak ortak dile çevrilir. Topluluklar arasında ikinci bir çeviri katmanı öngörülmez, çünkü topluluklar zaten aynı nesneler üzerinde çalışır.

Domain-Driven Design'da çeviri her topluluk sınırında yapılır ve yönü ilişkiye göre belirlenir. Alt taraf, ACL ile üst tarafın dilini kendi diline çevirir; Translation Map neyin alındığını ve neyin dışarıda bırakıldığını açıkça kaydeder. Asgari state ilkesi, her topluluğun diğerinin kavram dünyasından yalnızca ihtiyaç duyduğu kadarını almasını şart koşar.

Fark, çeviri kaybının kime ait olduğundadır. Ontology'de kayıp, ortak katmanı tanımlayan tarafın seçimiyle bir kez ve herkes için belirlenir. Domain-Driven Design'da her topluluk kendi kaybını kendisi seçer ve bu seçim bir eşleme tablosunda görünür kalır. Hibritleşme uyarısı ise Domain-Driven Design'ın kendi riskini de adlandırır: asgari state ilkesi terk edildiğinde sınır kendiliğinden bir kanonik şemaya doğru erir.

Altıncı Ayrım: Tutarlılığın Sınırı

Aggregate, Domain-Driven Design'ın "hangi kurallar aynı anda doğru kalmalı" sorusuna verdiği cevaptır. Bu soru bir performans ayarı değil, domain uzmanına sorulan bir sorudur: invariant cümlesi uzmana söylenir; tanınıyorsa sınırın içine, tanınmıyorsa dışına konur. Sınırın dışında kalan her şey için kabul edilebilir gecikme de yine uzmana sorulur ve ara durumlar (`REQUESTED`, `READY`, `FAILED`) modelde adlandırılır.

Bu çalışmanın incelediği Ontology kaynaklarında, Aggregate'e karşılık gelen ve tutarlılık sınırını bir domain kararı olarak tanımlayan bir kavram bulunamamıştır. Bu, platformun tutarlılık sağlamadığı anlamına gelmez. Düşündürdüğü şey, tutarlılığın nerede sağlanacağının bir domain kararı olarak değil, platformun işleyişine ait bir özellik olarak ele alındığıdır. Bu çıkarım ürün kaynaklarına dayandığı için, birinci karşı argümandaki kaynak asimetrisine tabidir.

Ayrımın sonucu birinci ayrımla aynı yöndedir. Domain-Driven Design'da "bu iki nesne aynı anda tutarlı olmak zorunda mı?" sorusu bir karar nesnesidir ve

cevabı kodda, Aggregate sınırı olarak görünür. Ontology'de aynı soru sorulabilir, ancak cevabın kaydedileceği bir model öğesi yoktur.

Karşı Argümanlar ve Tezin Sınırları

Bu bölüm, çalışmanın tezine yöneltilebilecek en güçlü itirazları ele almaktadır. İtirazların bir kısmı tezi zayıflatmakta, bir kısmı ise yalnızca kapsamını daraltmaktadır.

Birinci İtiraz: Kaynak Asimetrisi

İtiraz. Çalışma, Domain-Driven Design'ı öğretisiyle, Ontology'yi ise ürün sunumuyla karşılaştırmaktadır. Adil olan, ya iki öğretiyi ya da iki pratiği karşılaştırmaktır. Mevcut kurguda ilk yaklaşım en olgun hâliyle, ikincisi ise broşür hâliyle temsil edilmektedir.

Değerlendirme. İtiraz büyük ölçüde haklıdır ve yöntem bölümünde açıkça kabul edilmiştir. Kısmi bir savunma şudur: bir ürünün mimari kısıtları — kapsam zorunluluğu, ontolojiler arası bağ yasağı — pazarlama metni değil teknik gerçekliktir ve çalışmanın en güçlü çıkarımları bu kısıtlara dayanmaktadır. Buna karşın "gücün görünürlüğü" argümanının pratikte nasıl işlediği, saha gözlemi olmaksızın kesin biçimde savunulamaz. Aynı sınırlılık, Aggregate'in karşılığının bulunamadığı tespiti için de geçerlidir: bir kavramın ürün dokümantasyonunda bulunmaması, pratikte karşılığının bulunmadığını kanıtlamaz.

Tezin bu itiraz karşısındaki doğru statüsü şudur: sınanmayı bekleyen bir hipotez.

İkinci İtiraz: Kurumsallaşmış Ubiquitous Language

İtiraz. Çalışma, Ontology'de ortak dil inşasının bulunmadığını ima etmektedir. Oysa Palantir'in kurumsal pratiği — mühendisin uzun süre müşterinin yanında çalışması — Ubiquitous Language'in kurumsallaşmış bir biçimidir ve çoğu "DDD uyguluyoruz" beyanından daha fazla domain uzmanı teması içermektedir.

Değerlendirme. İtiraz güçlüdür ve çalışmanın kavram eşlemesi tablosunda "kısmi" olarak işaretlenen satırı destekler. Ancak tezi çürütmez, çünkü tez ortak dilin *kurulmadığını* değil, kurulan dilin *sahipliğinin* farklı olduğunu iddia etmektedir. Domain-Driven Design'da dil, sınır içindeki ekibe aittir ve o ekibin kod tabanında yaşar. Ontology'de dil, platformu yapılandıran tarafa aittir ve konfigürasyonda yaşar. Yoğun domain teması, bu sahiplik farkını ortadan kaldırmaz.

Üçüncü İtiraz: Çoklu Ontoloji İmkânı

İtiraz. Platform birden fazla ontolojiye izin vermektedir. Dolayısıyla "tek kanonik anlam katmanı" tasviri yanlıştır.

Değerlendirme. İtiraz teknik olarak doğrudur ve çalışma bu nedenle "karşılığı yok" iddiasını "farklı ölçekte" biçiminde revize etmiştir. Ancak itirazın gücü sınırlıdır: ontolojiler arasında bağ kurulamadığı için, çoklu ontoloji seçeneği ürünün temel değer önerisini fiilen iptal etmektedir. Sonuç, teknik olarak mümkün ancak ekonomik olarak cazibesiz bir seçenektir — ki bu, tasarımın yönlendirdiği davranışın tekil ontolojiye doğru olduğu anlamına gelir.

Dördüncü İtiraz: İdeal ile Pratiğin Kıyaslanması

İtiraz. Domain-Driven Design'ın kendi literatürü, uygulamaların büyük kısmının DDD Lite olduğunu kabul etmektedir. Bu durumda karşılaştırma, gerçekleşmeyen bir ideal ile fiilen çalışan bir ürün arasında yapılmaktadır. Pratikte Ontology, pratikte uygulanan Domain-Driven Design'dan daha fazla dilsel disiplin üretiyor olabilir.

Değerlendirme. Bu, tezin karşılaştığı en ciddi itirazdır ve çalışma onu çürütememektedir. Verilebilecek tek yanıt şudur: Domain-Driven Design'ın stratejik kısmı atlandığında bunun bir eksiklik olduğu literatürde adlandırılmış ve kayıt altına alınmıştır. Ontology'de aynı eksiklik için bir ad ve bir teşhis kriteri bulunmamaktadır. Yani fark, sonuçta değil, sonucun görünürlüğündedir — ki bu, çalışmanın merkezî tezinin kendisidir.

Buna karşın bu yanıt, karşılaştırmanın ampirik olarak sınanması gereğini ortadan kaldırmaz.

Beşinci İtiraz: Konfigürasyonun Yönetilebilirliği

İtiraz. Konfigürasyon değişikliklerinin izlenemez olduğu iddiası abartılıdır; platformlar sürüm kayıtları, yetkilendirme ve denetim izleri sunmaktadır.

Değerlendirme. İtiraz kısmen haklıdır. Çalışmanın iddiası izlenebilirliğin *teknik olarak imkânsız* olduğu değil, *sürtünmenin düşük* olduğudur. Kod değişikliği zorunlu bir inceleme sürecinden geçer ve tip sistemi tarafından denetlenir; konfigürasyon değişikliği için bu zorunluluk yapısal değil, kurumsal disipline bağlıdır. Fark niteliksel değil derecesel olmakla birlikte gerçektir.

Altıncı İtiraz: Taktiksel Maliyet

İtiraz. Domain-Driven Design'ın taktiksel yapı taşları ciddi bir uygulama maliyeti taşır: her kimlik ayrı bir tip, her tutar bir Value Object, her sınır ötesi etkileşim bir event, bir Outbox tablosu ve idempotent bir tüketici gerektirir. Ontology bu maliyetin büyük kısmını platforma devreder. Karşılaştırma, Domain-Driven Design'ın bu bedelini yeterince tartmamaktadır.

Değerlendirme. İtiraz haklıdır ve Domain-Driven Design literatürünün kendisi de bunu kabul eder: korunacak ciddi bir iş kuralı bulunmayan alanlarda CRUD tabanlı bir çözüm daha doğrudur (Ürkmez, 2026f); Domain Service'in

aşırı kullanımı modeli boşaltır (Ürkmez, 2026h); Aggregate kuralları belirli koşullarda bilinçli olarak esnetilebilir (Ürkmez, 2026i). Bu kabuller, ikinci ayrımdaki seçicilik bahsini güçlendirir: taktiksel maliyet, Core'a yoğunlaştırılmak üzere tasarlanmıştır. Ancak itirazın asıl gücü, maliyetin gerçek olması ve bu çalışmanın onu ölçmemiş olmasıdır.

Tezin Nihai Statüsü

Yukarıdaki itirazların ışığında, çalışmanın tezi şu daraltılmış biçimde savunulabilir:

> Ontology, kavramların topluluğa göreliliğini kabul eden ancak bu görelilikten doğan müzakereyi kurumsal olarak adlandırmayan bir tasarımdır. Domain-Driven Design ise aynı müzakereyi adlandırır ve bir karar nesnesine dönüştürür. Bu fark, iki yaklaşımın etkinliğine ilişkin bir hüküm değil, hangi sorunun sorulabilir kaldığına ilişkin bir tespittir.

Sonuç

Her iki yaklaşım da aynı gözlemden hareket etmektedir: aynı kelime farklı topluluklarda farklı bir nesnedir ve bunu tek bir şemaya sıkıştırmak modeli bozar.

Cevapları ise ayrışmaktadır.

Domain-Driven Design, sınırları çizmeyi, her sınırın içini dilsel olarak tutarlı tutmayı, sınırlar arasındaki güç ilişkisini açıkça adlandırmayı ve çeviri maliyetini kabul etmeyi önerir. Bedeli, sınırlar arası sorgulamanın pahalı olması ve koordinasyonun sürekli bir iş hâline gelmesidir.

Ontology, ortak bir nesne katmanı kurmayı, farklılığı bu katmanın üzerinde nesne tipleriyle ifade etmeyi ve tüm birimlerin aynı grafik üzerinde çalışmasını önerir. Bedeli, ortak katmanı tanımlayan tarafın fiilî üstünlük kazanması ve bu üstünlüğün açık bir tasarım kararı olarak kaydedilmemesidir.

İkisi de aynı gerilimin iki ucudur: **anlamın yerel olması ile sistemin bütünsel olarak sorgulanabilir olması arasındaki gerilim.** Bu gerilim çözülmüş değildir; her yaklaşım bir ucu seçmekte ve diğerinin bedelini ödemektedir.

Taktiksel düzeydeki inceleme bu gerilimi daha küçük bir ölçekte yeniden göstermiştir. İki yaklaşım nesne, özellik ve fonksiyon düzeyinde büyük ölçüde örtüşür; ayrıştıkları yer, sınır çizen ve sınırlar arası ilişkiyi adlandıran araçlardır. Bu araçlar büyük ölçekte Bounded Context ve Context Map, küçük ölçekte Aggregate ve ACL'dir. Domain-Driven Design'ın taktiksel kararlarının her biri domain uzmanına geri götürülen bir cümle sınamasına dayandığı için, formalizasyonun pratiğe bağımlılığı sistemin içinde sürekli işleyen bir süreç olarak korunur. Wittgenstein'ın uyarısı bu yolla çözülmez, ama unutulmaz.

Çalışmadan çıkan uygulamaya dönük sonuç şudur: bir kurumda ontoloji kuruluyorsa, Context Map'in yine de çizilmesi gerekmektedir. Haritadaki tüm oklar aynı yöne bakacaktır; belirleyici olan, bunun bilinçli bir tercih mi yoksa varsayılan bir sonuç mu olduğunun bilinmesidir. Aynı ilke küçük ölçekte de geçerlidir: hangi iki nesnenin aynı anda tutarlı olması gerektiği, platform ne olursa olsun, domain uzmanıyla birlikte açıkça kararlaştırılmalıdır.

Özet formülasyon:

> Ontology, Domain-Driven Design'ın stratejik yarısını atlayıp taktiksel yarısını platform seviyesine çıkarma girişimidir. Kazandığı şey bağlantılanabilirlik, kaybettiği şey sınır müzakeresinin ve tutarlılık kararının görünürlüğüdür.

İleri Araştırma Yönleri

Çalışmanın ampirik dayanaktan yoksun olması, dört somut araştırma sorusu doğurmaktadır.

1. Ontoloji kuran kurumlarda, nesne tipi tanımlarının hangi birim tarafından ve hangi müzakere süreciyle belirlendiğinin saha gözlemiyle incelenmesi.

2. Kanonik şemadan ontolojiye geçiş yapan kurumlarda, birimlerin kayıt dışı tablo tutma davranışının azalıp azalmadığının ölçülmesi.

3. Stratejik tasarımı uygulayan ve uygulamayan ekiplerde, kavramsal bulanıklığa bağlı hataların sıklığının karşılaştırılması.

4. Ontoloji kuran kurumlarda, birimler arası tutarlılık gereksinimlerinin — hangi iki nesnenin aynı anda tutarlı olması gerektiğinin — nerede ve kim tarafından kararlaştırıldığının incelenmesi.

Kaynakça

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.
- Fowler, M. (2003). "Anemic Domain Model". martinfowler.com.
- Palantir Technologies. *Ontology Overview*. Foundry Dokümantasyonu.
- Palantir Technologies. *Ontologies Overview*. Foundry Dokümantasyonu.
- Ürkmez, Z. (2026a). "DDD #1: Tasarım ve Yazılımın Sonunda Buluştuğu Yer". Codewarts Bülten, 21 Ocak 2026.
- Ürkmez, Z. (2026b). "DDD #2: Anlamın Sınırları". Codewarts Bülten, 3 Şubat 2026.
- Ürkmez, Z. (2026c). "DDD #3: Modellemeye Başlıyoruz, Bir E-Ticaret Domain'ini Sıfırdan Okumak". Codewarts Bülten, 2 Mart 2026.
- Ürkmez, Z. (2026d). "DDD #4: Context Map, Bounded Context'ler Arasındaki Güç Dengesi". Codewarts Bülten, 21 Temmuz 2026.
- Ürkmez, Z. (2026e). "DDD #5: Sınırda Çeviri, ACL, Open Host Service ve Eventual Consistency". Codewarts Bülten, 28 Temmuz 2026.
- Ürkmez, Z. (2026f). "DDD #6: Tactical Design Başlıyor, Entity ve Kimlik Meselesi". Codewarts Bülten, 4 Ağustos 2026.
- Ürkmez, Z. (2026g). "DDD #7: Value Object, Kimliği Olmayan Kural Taşıyıcısı". Codewarts Bülten, 12 Ağustos 2026.
- Ürkmez, Z. (2026h). "DDD #8: Domain Service, DDD'nin En Riskli Yapı Taşı". Codewarts Bülten, 26 Ağustos 2026.
- Ürkmez, Z. (2026i). "DDD #9: Aggregate, Tutarlılığın Sınırı". Codewarts Bülten, 8 Eylül 2026.
- Ürkmez, Z. (2026j). "DDD #10: Domain Event, Aggregate Sınırının Dışına Taşanlar". Codewarts Bülten, 16 Eylül 2026.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Wittgenstein, L. (1921/2006). *Tractatus Logico-Philosophicus*. Çev. O. Aruoba. Metis Yayınları.
- Wittgenstein, L. (1953/2007). *Felsefi Soruşturmalar*. Çev. H. Barışcan. Metis Yayınları.